Name: Naveen Kumar P **Reg. No.:** 192412326

Experiment No.: 6

Experiment Question

Write a program to implement the Naïve Bayes algorithm in Python and to display the results using a confusion matrix and accuracy.

Aim

To implement the Naïve Bayes classifier in Python on the Breast Cancer dataset, and to evaluate its performance using a confusion matrix and accuracy.

Concept

Naïve Bayes is a supervised classification algorithm based on **Bayes' theorem**. It calculates the probability of each class for a given sample and predicts the class with the highest probability. It is called "naïve" because it assumes that all features are independent of each other given the class.

When the features are continuous, the **Gaussian Naïve Bayes** variant is used. It assumes that the values of each feature in each class follow a normal (Gaussian) distribution, whose mean and variance are estimated from the training data.

A **confusion matrix** is a table that compares the actual classes with the predicted classes. For a two-class problem it shows the true positives (TP), false negatives (FN), false positives (FP) and true negatives (TN). **Accuracy** is the fraction of all predictions that are correct.

Dataset Description

The Breast Cancer Wisconsin (Diagnostic) dataset, which is built into scikit-learn, is used. It has 569 samples and 30 numeric features computed from images of breast tumour cells (radius, texture, perimeter, area, smoothness and so on). The target has two classes: malignant (212 samples) and benign (357 samples).

The data is split into 80% training (455 samples) and 20% testing (114 samples), with the class proportions kept the same in both parts (`random_state = 42`).

Algorithm

Gaussian Naive Bayes Algorithm
Training:
1. For each class c, calculate the prior probability P(c) = (samples in c) / (total samples).
2. For each class c and each feature x_i, calculate the mean and the variance of x_i
   over the training samples of class c.

Prediction for a new sample x = (x_1, ..., x_n):
3. For each class c, calculate the posterior score:
      P(c) * P(x_1 | c) * P(x_2 | c) * ... * P(x_n | c)
   where each P(x_i | c) comes from the Gaussian probability density function.
4. Predict the class with the highest posterior score.

Step-by-step Procedure

1. Import the required libraries and load the Breast Cancer dataset.
2. Print the dataset shape, the class names and the number of samples in each class.
3. Split the data into training and testing sets (80% / 20%).
4. Create a Gaussian Naïve Bayes model and train it on the training set.
5. Predict the classes of the test samples.
6. Compute the confusion matrix by comparing the predicted classes with the actual classes.
7. Compute the accuracy and the classification report (precision, recall, F1-score).

Formulae

Bayes' theorem:

$$P(c \mid x) = \frac{P(x \mid c)\, P(c)}{P(x)}$$

Naïve Bayes classification rule (features assumed independent given the class):

$$\hat{y} = \arg\max_{c}\; P(c) \prod_{i=1}^{n} P(x_i \mid c)$$

Gaussian likelihood of feature x_i in class c, with mean μ and variance σ²:

$$P(x_i \mid c) = \frac{1}{\sqrt{2\pi\sigma_c^{2}}} \exp\!\left(-\frac{(x_i - \mu_c)^2}{2\sigma_c^{2}}\right)$$

Accuracy, precision and recall from the confusion matrix:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

Python Program

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report

# Step 1: Load the dataset
data = load_breast_cancer()
X, y = data.data, data.target
names = [str(n) for n in data.target_names]
print("Dataset shape:", X.shape)
print("Classes:", names)
print("Samples per class:", {name: int((y == i).sum()) for i, name in enumerate(names)})

# Step 2: Split into training and testing sets (80% / 20%)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print("\nTraining samples:", len(X_train))
print("Testing samples :", len(X_test))

# Step 3: Train the Gaussian Naive Bayes classifier
model = GaussianNB()
model.fit(X_train, y_train)

# Step 4: Predict on the test set
y_pred = model.predict(X_test)

# Step 5: Evaluate using confusion matrix and accuracy
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:")
print(cm)
print("\nAccuracy: {:.2f}%".format(accuracy_score(y_test, y_pred) * 100))
print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=names))
```

Output

```
Dataset shape: (569, 30)
Classes: ['malignant', 'benign']
Samples per class: {'malignant': 212, 'benign': 357}

Training samples: 455
Testing samples : 114

Confusion Matrix:
[[38  4]
```

```
 [ 3 69]]

Accuracy: 93.86%

Classification Report:
              precision    recall  f1-score   support

   malignant       0.93      0.90      0.92        42
      benign       0.95      0.96      0.95        72

    accuracy                           0.94       114
   macro avg       0.94      0.93      0.93       114
weighted avg       0.94      0.94      0.94       114
```

Reading the confusion matrix (rows = actual class, columns = predicted class):

	Predicted malignant	Predicted benign
Actual malignant	38	4
Actual benign	3	69

Result / Conclusion

The Gaussian Naïve Bayes classifier was implemented in Python and tested on 114 samples. It classified 38 malignant and 69 benign samples correctly (107 of 114), with 4 malignant samples predicted as benign and 3 benign samples predicted as malignant. The accuracy is **93.86%**. Hence the Naïve Bayes algorithm was successfully implemented and evaluated using the confusion matrix and accuracy.